COMPUTER ENGINEERING

QUEZON CITY

COURSE CODE/TITLE:	CPE 010-CPE21S4	SECTION:	CPE21S4
DATE PERFORMED:	15/10/2026	DATE SUBMITTED:	15//010/2026
NAME:	Jullian Edrain R. Sevilla	INSTRUCTOR:	Engr. JIMLORD QUEJADO

ACTIVITY NO. ___ [ACTIVITY NAME]

1. PROCEDURE OUTPUT

What procedures did you perform?

Strategy	Algorithm	Analysis
Recursion	Factorial / Fibonacci Sequence	Defines a problem in terms of itself with a base case. For factorial: n! = n × (n-1)!; base case 0! = 1. Each call reduces the input size, moves toward the base case, and returns values up the chain. Key parts: base case, work toward a simpler problem, recursive call.
Brute Force	Linear Search	Checks every element one by one, no shortcuts, no skipping. Tries all possibilities unconditionally until it finds the target or exhausts all options. Always correct but not efficient for large data — time complexity O(n).
Backtracking	N-Queens Problem	Builds solutions step by step. If a partial solution violates rules, it discards that path immediately (“backtrack”) and tries the next choice. Uses recursion to explore a state-space tree and prunes

		invalid branches early to save time.
Greedy	Fractional Knapsack / Coin Change	At every step, it picks the best immediate option — the locally optimal choice — without looking ahead. Fast and simple, but does not guarantee the best overall answer. Example: Min Steps to One with n=10 → greedy gives 4 steps; optimal is 3 steps.
Divide-and-Conquer	Merge Sort	Breaks the array into halves recursively, sorts each half separately, then merges the sorted halves back together. Three phases: Divide into smaller subproblems → Solve each subproblem → Combine results into final answer. Subproblems are independent and identical to the original problem.

CPE010_HOA3p1_JullianSevilla_sc1.cpp

```cpp
#include <iostream>
#include <vector>
#include <algorithm>
using namespace std;

vector<int> memo;

int getMinStepsMemo(int n) {
    if (n == 1) return 0;
    if (memo[n] != -1) return memo[n];

    int r = 1 + getMinStepsMemo(n - 1);
    if (n % 2 == 0)
        r = min(r, 1 + getMinStepsMemo(n / 2));
    if (n % 3 == 0)
        r = min(r, 1 + getMinStepsMemo(n / 3));

    memo[n] = r;
    return r;
}

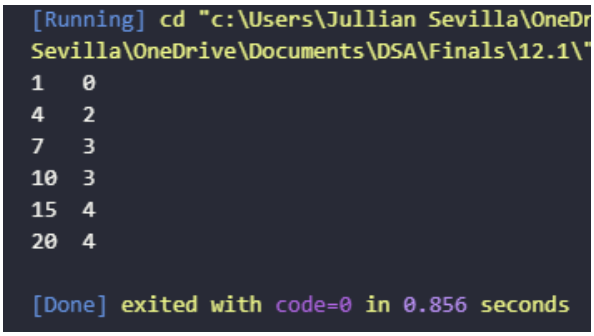
int main() {
    int testValues[] = {1, 4, 7, 10, 15, 20};
    for (int n : testValues) {
        memo.assign(n + 1, -1);
        cout << n << "\t" << getMinStepsMemo(n) << endl;
    }
    return 0;
}
```

```
[Running] cd "c:\Users\Jullian Sevilla\OneD
Sevilla\OneDrive\Documents\DSA\Finals\12.1\
1   0
4   2
7   3
10  3
15  4
20  4

[Done] exited with code=0 in 0.795 seconds
```

This code turns a number into 1 in as few moves as possible by either subtracting 1 or dividing by 2 or 3. It uses a special list to remember past answers so it does not have to keep recalculating the math over and over. For 1, the answer is 0 because it is already at 1. For 4, it takes 2 moves; for 7 and 10 it takes 3 moves; and for 15 and 20 it takes 4 moves.





This code does the same thing to get a number down to 1, but it starts at 1 and counts up using a loop instead of starting from the big number. It fills up a list with the best answers starting from 1, then 2, then 3, all the way up to the target number. It still subtracts 1 or divides by 2 or 3 to find the fewest moves. The final answers are the same, so 1 takes 0 moves, 4 takes 2 moves, 7 and 10 take 3 moves, and 15 and 20 take 4 moves.

2. ANSWERS TO SUPPLEMENTAL ACTIVITY

Copy and paste the questions from the lab and include your answers after.

CPE010_HOA3p1_JullianSevilla_sc1.cpp CPE010_HOA3p1_JullianSevilla_sc2.cpp
CPE010_HOA3p1_JullianSevilla_sc3.cpp > main()

```cpp
#include <iostream>
#include <vector>
using namespace std;

int countPaths(vector<vector<int>>& mat, int m, int n, int cost) {
    if (cost < 0) return 0;
    if (m == 0 && n == 0) return (mat[0][0] == cost) ? 1 : 0;
    if (m == 0) return countPaths(mat, 0, n - 1, cost - mat[m][n]);
    if (n == 0) return countPaths(mat, m - 1, 0, cost - mat[m][n]);

    return countPaths(mat, m - 1, n, cost - mat[m][n]) +
           countPaths(mat, m, n - 1, cost - mat[m][n]);
}

int main() {
    vector<vector<int>> matrix = {
        {4, 7, 1, 6},
        {6, 7, 3, 9},
        {3, 8, 1, 2},
        {7, 7, 3, 4}
    };
    int targetCost = 25;
    int M = matrix.size(), N = matrix[0].size();

    cout << "Matrix:\n";
    for (auto &row : matrix) {
        for (int v : row) cout << v << " ";
        cout << "\n";
    }
```

```
[Running] cd "c:\Users\Jullian Sevilla\OneDr
Sevilla\OneDrive\Documents\DSA\Finals\12.1\"
Matrix:
4 7 1 6
6 7 3 9
3 8 1 2
7 7 3 4

Target Cost: 25
Number of paths: 0

[Done] exited with code=0 in 0.796 seconds
```

This code tries to find paths through a grid of numbers from the top-left to the bottom-right. It adds up all the numbers along each path to check if the total sum equals the target cost of 25. It works backward using recursion to check every possible move. The final output prints the grid and target cost, showing that there are 0 paths because no path adds up to exactly 25.

3. ARTIFICIAL INTELLIGENCE (AI) USE DISCLOSURE STATEMENT

I hereby declare that artificial intelligence (AI) tools were used, if applicable, in the preparation of this academic work. The following indicates the nature and extent of AI assistance:

Tools Used: Grammarly, Copilot

Examples: ChatGPT, Microsoft Copilot, Grammarly, QuillBot, etc.

Purpose of Use: Grammar correction, code assistance, and explanation of concepts

Examples: grammar correction, idea generation, formatting, summarization, code assistance, explanation of concepts, etc.

Extent of Use: Did not replace the original thinking

COMPUTER ENGINEERING

QUEZON CITY

Explain how AI was used and confirm that it did not replace original thinking, analysis, authorship, or completion of the required work.

By submitting this activity, I affirm that my use of AI complies with T.I.P.'s AI usage policy and does not exceed the permitted limits for similarity, automated content generation, or academic assistance.

I understand that any false, incomplete, or misleading disclosure may be subject to investigation in accordance with due process and may result in sanctions for violation of existing T.I.P. policies.

Lab activities, templates, and related instructional materials shall not be reuploaded, reproduced, distributed, or shared with external organizations or third parties without the prior express written permission of the Technological Institute of the Philippines.